

MET CS688

WEB ANALYTICS AND MINING

ZLATKO VASILKOSKI

AI AGENTS IN PYTHON

Introduction

"What I'm seeing with AI agents is an exciting trend that I believe everyone building AI should pay attention to…"

Andrew Ng, Co-founder of Google Brain, former Chief Scientist at Baidu, founder of Coursera

Andrew Ng is starting a Python course for Beginners who wants to learn AI.

Creating Autonomous AI Agents from Scratch

- In this presentation, we will create an autonomous AI agent using Python from scratch.
- We will not use any third-party libraries like DSPy, Langchain or CrewAI.
- We will focus on a basic AI agent example to understand the structure and development process of AI agents.



What are the limitations of LLMs?

If we prompt a good LLM like Llama with an easy question such as:

What is the current temperature in Boston?

Most likely the answer will not be correct. Why?

One of the main limitations of LLMs is that it can't access **real time data**.

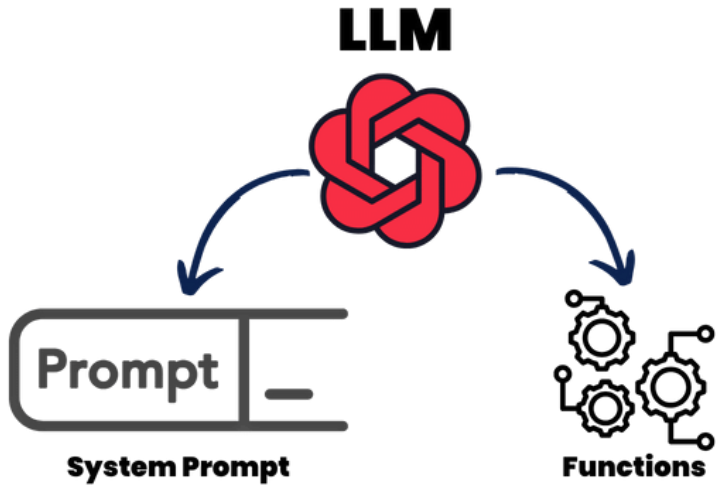
They only generate responses based solely on pre-existing training data.

A solution to this are Autonomous AI Agents.

Understanding the limitations of Large Language Models (LLMs)

- LLMs cannot access real-time data and only generate responses based on pre-existing training data.
- This limitation can be overcome by integrating LLMs with external functions and enhanced prompting mechanisms.
- **Autonomous AI agents can integrate LLMs with external functions to provide more accurate and up-to-date responses.**

What is an AI Agent?



An Autonomous AI Agent integrates a

- **large language model** (LLM) with
- **external functions** and
- **enhanced prompting** mechanisms.

At their core, AI agents are autonomous software entities that can

- perceive their environment,
- process information,
- and take actions to achieve specific goals or perform designated tasks.

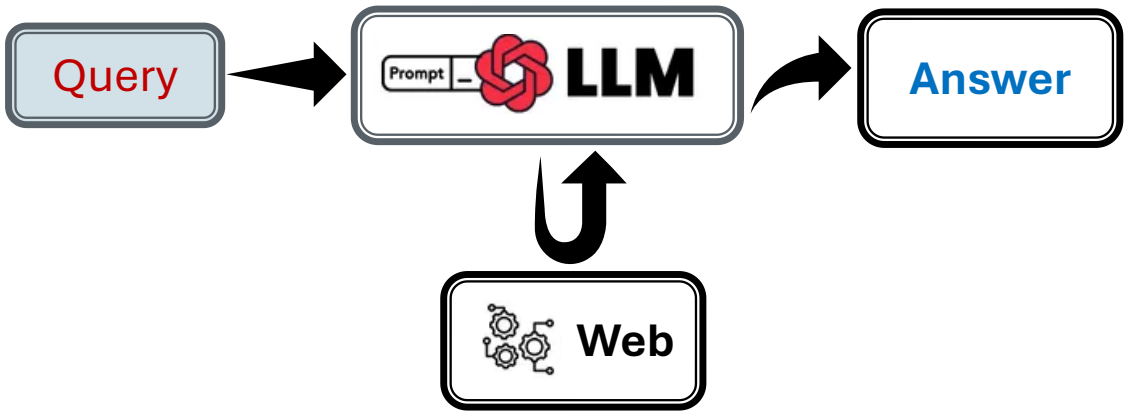
Unlike traditional software systems that rely on predefined rules and explicit programming, AI agents are designed to operate with a higher level of independence and adaptability.

They can learn from their experiences, adjust their behavior based on feedback, and make decisions in real-time to optimize the solution.

Autonomous AI Agent Architecture

Overview of the Autonomous AI Agent Architecture

- The AI agent integrates an LLM with external functions and enhanced prompting mechanisms.
- In the core of the AI Agent Architecture are LLM is powered by a **ReAct System Prompt** that processes the user query
  - **ReAct** allows it to think about the user query (called ***Thought***), understands it, then decides how to answer it and picks an action if needed.
  - If external action is needed it selects and executes an external **user defined function**.
  - The function result is returned back to the LLM.
  - Based on the result LLM generates a response.
- The **ReAct system** prompt enables the LLM to think about the user query, understand it, decide how to answer, and pick an action if needed.



Autonomous AI Agent Architecture

Defining the **ReAct System Prompt**

- The ReAct prompt instructs the LLM to think about the user query, understand it, decide how to answer, and pick an action if needed.
- The prompt guides the LLM through a cycle of Thought, Action, and Response.
- The loop structure (Thought, Action, PAUSE, Action_Response) enables the LLM to handle user queries effectively.

The ReAct Prompt is what enables the AI agent to mimic human behavior.

- This system prompt guides the model through a cycle of Thought, Action, and Response, allowing it to handle user queries effectively.
  - Thought: Understand and interpret the query.
  - Action: Select and execute the appropriate function from the available actions.
  - Action Response: Use the result from the action to formulate the response.

- Integrating the ReAct System Prompt with the AI Agent
  - Add the ReAct system prompt configuration to prompts.py.
  - The prompt instructs the LLM to run within a loop of Thought, Action, and Action_Response.
  - The loop structure guides the LLM to understand the user query, take an action, and use the outcome to respond.

Tools & Concepts

LangGraph is a powerful extension of the LangChain library, designed to help build advanced AI agents.

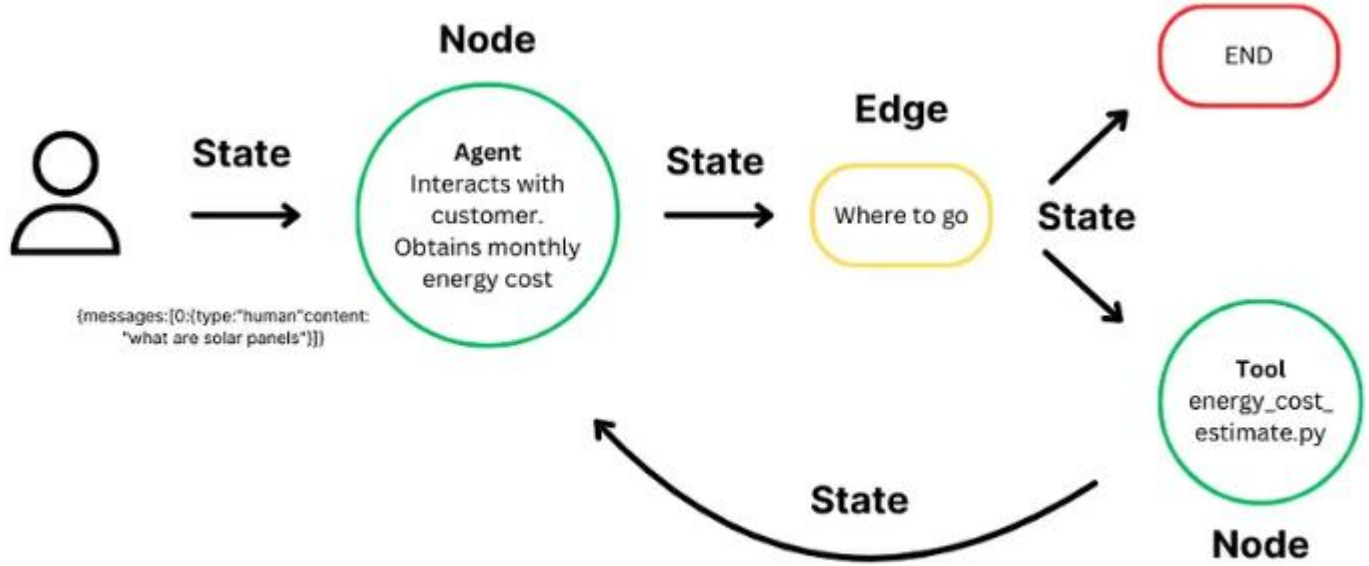
LangGraph is an advanced library built on top of LangChain,

- Designed to enhance your Large Language Model (LLM) applications by introducing cyclic computational capabilities.
- While LangChain allows the creation of Directed Acyclic Graphs (DAGs) for linear workflows, LangGraph takes this a step further by enabling the addition of cycles, which are essential for developing complex, agent-like behaviors.
- These behaviors allow LLMs to continuously loop through a process, dynamically deciding what action to take next based on evolving conditions.

Tools & Concepts

At the heart of LangGraph is the concept of a **stateful graph**:

- **State**: Represents the context or memory that is maintained and updated as the computation progresses. It ensures that each step in the graph can access relevant information from previous steps, allowing for dynamic decision-making based on accumulated data throughout the process.
- **Nodes**: Serve as the building blocks of the graph, representing individual computation steps or functions. Each node performs a specific task, such as processing inputs, making decisions, or interacting with external systems. Nodes can be customized to execute a wide range of operations within the workflow.
- **Edges**: Connect nodes within the graph, defining the flow of computation from one step to the next. They support conditional logic, allowing the path of execution to change based on the current state and facilitate the movement of data and control between nodes, enabling complex, multi-step workflows.



Langraph: nodes, states, and edges.

Lab Project: Create AI Agent in Python

Task: Created Autonomous AI Agents using Python, without any third-party libraries like Langchain, CrewAI or DSPy.

- The agent integrates an LLM with external functions and enhanced prompting mechanisms.
- The ReAct system prompt enables the LLM to think about the user query, understand it, decide how to answer, and pick an action if needed.

- Set up the Python Project
  - Create a new Python project.
    - Create virtual environment
    - Install packages (groq, requests, bs4)

  - Create three Python files:

```
# Using Groq
model_to_use, api_key = 'llama3-70b-8192', 'GROQ_API_KEY'
client = Groq(api_key=os.environ.get(api_key))
```

    - **main.py**
      - Initialize LLM from open-sources such as Groq instead of OpenAI for which you need to pay.
      - Contains the Agent Thought, Action, and Response cycle that processes user inputs and handles AI responses.
    - **actions.py**
      - Contains user defined functions (actions) for the AI Agent
    - **prompts.py**
      - Contains **system prompt** with specifics (**available actions** and **action response**) for the LLM.

Lab Project: Create AI Agent in Python

- Creating user defined functions (actions) for the AI Agent in actions.py
  - Define a function to go to the weather.com ang get the weather conditions for a city.
  - This function serves as a basic example to demonstrate how the agent can utilize external functions.
  - The function returns fixed response (weather conditions) at the time of request for the provided city.

Task 1:

- Use the following link (Note: Since scraping is not desirable as alternative get API key from Open Weather and use it)
  https://www.geeksforgeeks.org/how-to-extract-weather-data-from-google-in-python/
- To create an action function called
  "get_current_temperature(city_name)".
- Save this function in "**actions.py**"
  - The argument of this function is a city name (i.e. "Boston")
  - It returns weather data (temperature, time, sky)
  - To confirm the temperature, visit site:
    'https://www.google.com/search?q=weatherBoston'

```
def get_current_temperature(city_name):
    # enter city name
    city = city_name

    # creating url and requests instance
    url = "https://www.google.com/search?q=" + "weather" + city
    html = requests.get(url).content

    # getting raw data
    soup = BeautifulSoup(html, features: 'html.parser')
    temperature = soup.find( name: 'div', attrs={'class': 'BNeawe iBp4i AP7Wnd'}).text
    str = soup.find( name: 'div', attrs={'class': 'BNeawe tAd8D AP7Wnd'}).text

    # formatting data
    data = str.split('\n')
    time = data[0]
    sky = data[1]

    # getting all div tag
    listdiv = soup.findAll( name: 'div', attrs={'class': 'BNeawe s3v9rd AP7Wnd'})
    strd = listdiv[5].text

    # getting other required data
    pos = strd.find('Wind')
    other_data = strd[pos:]

    return temperature, time, sky, other_data
```

Lab Project: Create AI Agent in Python

Task 2:

- Integrate the ReAct System Prompt with the AI Agent as a simple **while loop**.
  - Add the ReAct system prompt configuration to **prompts.py**.
  - The prompt instructs the LLM to run within a loop of Thought, Action, and Action-Response.
  - The loop structure guides the LLM to understand the user query, take an action, and use the outcome to respond.
  - Create the following prompts:
    - available_actions – Instructs the LLM what are doing the user defined functions (actions).
    - action_response – Illustrates to the LLM what are the function names & arguments.
    - system_prompt – Combines everything into a ReAct System Prompt for the AI Agent.

```python
while turn_count < max_turns:
    print(f"Loop: {turn_count}")
    print("----------------------")
    turn_count += 1

    response = generate_text_with_conversation(messages, model=model_to_use)

    print(response)

    json_function = extract_json(response)

    if json_function:
        function_name = json_function[0]['function_name']
        function_parms = json_function[0]['function_parms']
        if function_name not in available_actions:
            raise Exception(f"Unknown action: {function_name}: {function_parms}")
        print(f" -- running {function_name} {function_parms}")
        action_function = available_actions[function_name]
        # call the function
        result = action_function(**function_parms)
        function_result_message = f"Action_Response: {result}"
        messages.append({"role": "user", "content": function_result_message})
        print(function_result_message)
    else:
        break
```

Lab Project: Create AI Agent in Python

Task 3:

Having established the ReAct System Prompt and defined the necessary functions, integrate these elements to construct the AI agent in **main.py**.

- List the functions the agent can utilize

```python
# Define Available Functions
available_actions = {
    "get_stock_price": actions.get_stock_price,
    "get_current_temperature": actions.get_current_temperature
}
```

- Define the user prompt and the messages that will be passed to the generate_text_with_conversation function.

```python
user_prompt = "What is the current weather in Boston?"

messages = [
    {"role": "system", "content": prompts.system_prompt},
    {"role": "user", "content": user_prompt},
]
```

```python
def generate_text_with_conversation(messages, model=model_to_use):
    response = client.chat.completions.create(
        model=model,
        messages=messages
    )
    return response.choices[0].message.content
```

- Implement the Agent's Thought, Action, and Response Cycle that processes user inputs and handles AI responses.
  - The prompt instructs the LLM to run within a loop of Thought, Action, and Action-Response.
  - The loop structure guides the LLM to understand the user query, take an action, and use the outcome to respond.

Lab Project: Create AI Agent in Python

Task 4:

Note the difference when you ask your AI Agent different prompts such as:

- user_prompt = "What is the current temperature in Boston?"
  user_prompt = "What is the current weather in Boston?“
- Different city etc.

Make appropriate changes in all the 3 python script files to add additional AI Agent that will get the current stock price.

- Use yahoo finance and this function

```
def get_stock_price(stock):
    symbol = yf.Ticker("META")
    current_price = symbol.fast_info.last_price
    return current_price
```

Result:

```
====================================
Current price for stock symbol META is 703.5700073242188
====================================
Loop: 1
----------------------
Thought: I should check the current stock price for META.

Action:
{
  "function_name": "get_stock_price",
  "function_parms": {
    "stock_symbol": "META"
  }
}

PAUSE
 -- running get_stock_price {'stock_symbol': 'META'}
Action_Response: 703.5700073242188
Loop: 2
----------------------
Answer: The current stock price of META is 703.57.
```

A

- A